

BEGINNER-FRIENDLY PDF EDITION

Ontology Machine Quickstart

The short path into The Ontology Machine.

Core model: the Artifact Tree preserves evidence; the Corpus DB makes it queryable; the Frontend lets you talk to the corpus; the Orchestrator runs and debugs ingestion.

Prepared from Quickstart_Handbook.md. Revised for readability, clean tables, styled callouts, and rendered flowcharts.

Contents

1. First read: cost, scope, and entry model	3
2. The First Five Minutes	6
3. The Bundled Demo	8
4. What To Configure	9
5. The Three Main Surfaces	10
6. Optional Extractor Tools	11
7. Which Agent Do I Use?	12
8. When To Use The Orchestrator	14
9. The Artifact Tree In One Minute	15
10. The Corpus DB In One Minute	16
11. What Is A Semantic Release?	17
12. What Is An Ontology Lens?	18
13. Create Your Own Corpus	19
14. First Useful Demo Session	20
15. Common First Problems	21
16. What To Read Next	23

Recommended first pass for new users: read sections 1-8, then use the creation route or troubleshooting section that matches your task.

1. First read: cost, scope, and entry model

This is the short path into The Ontology Machine.

Very important first notice

If you work with the Ontology Agent, be aware that this work is quite token intensive. It is not uncommon for a single chat turn to generate 5 background model calls and 15 tool calls, depending on the agent task. Depending on the size of the database you work with, those model calls and tool outputs can easily reach up to 150k tokens in a single user request. There is a token count estimate beside the **Send** button that accumulates token usage estimates throughout a single chat session, so you can see how many tokens have already gone into the work.

This is not a bug. This is how the Ontology Agent works and creates its responses. It needs those tokens to be effective. If you work through the API, choose your model carefully and always know the actual API costs so you do not accidentally fry your bank account.

A database can require many read rounds before the Agent has enough evidence to write a useful lens batch and to create a whole lens that covers the whole corpus, you might need dozens of chat turns with hundreds of tool calls. Keep that in mind (what a lens is will be explained further down).

If you intend to not only use this software for query tasks but for serious ontology work, I strongly suggest to not exceed a database size of 100, max 200 pages. Otherwise your knowledge mining will remain quite shallow since the token usage directly correlates with DB size and coverage intent.

A small but highly correlated corpus will yield way more insight than a large one that is only superficially scratched by the Ontology Agent.

If you use your OpenAI OAuth subscription, be aware that this will drain your quota much faster than a normal chat. This also partially applies to the Query Agent, but to a lesser degree, since it is less token-hungry and has hard caps. The Ontology Machine cannot predict or control provider billing, subscription quotas, rate limits or future price changes. Always check your provider account and pricing before running large ontology work.

If you want to be cost effective, I would suggest creating an OpenRouter account and charging it with 10 bucks or so. Then you get 1000 responses per day free of charge with their free models, and you can choose the free Owl Alpha model, which has a 1 million context window. It is not as powerful as the frontier models from the big four, but it is free and does the job, more or less. Rather less the

longer the chat session runs and suffers from being a free model that is subjected to the whims of OpenRouter, which means provider errors are common but transient.

Alternatively, you can use DeepSeek V4, which is behind the frontier models when it comes to stability in tool use over long contexts but still capable for light ontology work but not for running The Machine since it is text-only. It is dirt cheap compared to the big four. If you use OpenRouter, you can also use OpenAI's embedding-3-small model for your embeddings, which is only 2 cents per million input tokens and has no output token cost.

However, be aware that certain operations need a vision-capable model. If you use OpenRouter, you have to make sure yourself that the model you choose is indeed a vision model. Also, Agent work needs tool call capability AND follow the instruction to actually use them. Owl Alpha for example is a tool capable model but very hesitant to use them if the context window is already large.

Also, it is not guaranteed that all models or all listed model providers within The Machine will work. Some model calls demand specific API output schemas, and the model you choose must support those too. It is up to the user to play around and find out.

As a side note: This software is an excellent benchmark for model capabilities in real world use cases. As of June 2026, GPT 5.4 and 5.5 are the models that perform the best here since what this software is testing is instruction following and hard gated tool use. In this domain, other models like DeepSeek V4 Pro falls far behind and this software basically exceeds the capability of it at times.

The flagship models of Anthropic and Google are close, but not on par with OpenAI. That's the reason why the default go-to is GPT 5.4 and GPT 5.4 mini. Using 5.5 is overkill and way too expensive even with a subscription.

The Machine is not a normal chat app and not just a document converter. It is a local evidence machine. It takes source files, renders and extracts them, validates what the model said, normalizes the result through a semantic release, materializes everything into a SQLite corpus DB, and then lets agents query, inspect, mine and extend that corpus without losing the evidence trail.

The important thing to understand first:

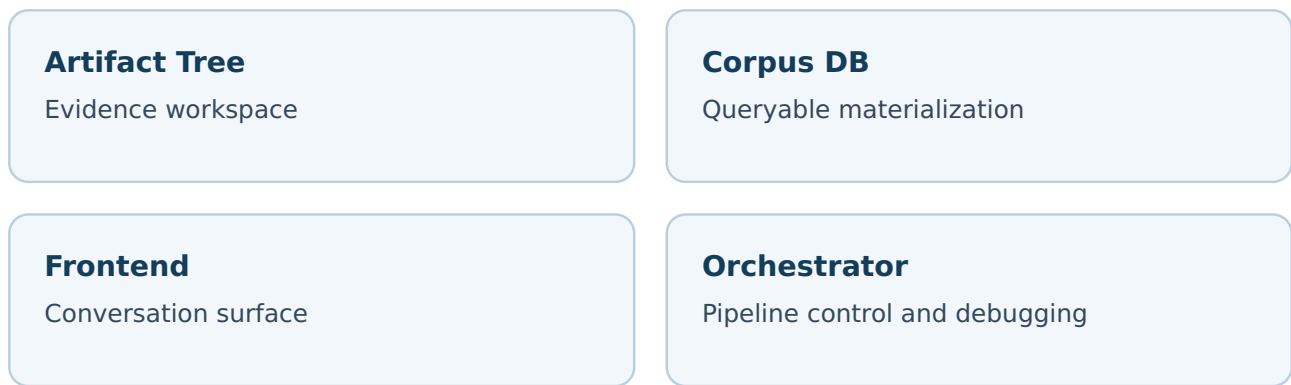


Figure 1. The minimum entry model: evidence workspace, queryable DB, chat surface, and pipeline control.

That is the whole entry model.

2. The First Five Minutes

If you are looking at the source workspace, the useful launchers are:

```
Client Frontend\config.bat  
Client Frontend\start.bat  
00 - Orchestrator\run.bat
```

Start the config UI first:

```
Client Frontend\config.bat
```

It opens the configuration page, usually here:

```
http://127.0.0.1:3001/config
```

Then start the main chat UI:

```
Client Frontend\start.bat
```

It opens the working frontend, usually here:

```
http://127.0.0.1:3000
```

The config server and the chat server are different processes. If port `3001` is open but the agents do not answer, you probably started only the config server. Start `Client Frontend\start.bat` as well.

Operator note

Important: If something goes wrong with the Taxonomy Agent, it is generally better to reset the Kernel than to try to convince the Agent to proceed with whatever the error caused. Resuming from an error caused by a misclick or folder confusion is something the Kernel does not handle well.

Also, if you want to use the extractor tools, make sure you run the “Install Optional Trafilatura” script within the “Article Archive Extractor” folder and the “Install yt-dlp” script in the “YouTube Transcript Extractor” folder. Those scripts install a better text extractor for websites and the yt-dlp subtitle extractor. The standard

HTML extractor remains available as a fallback. Without those installs, you will still be able to extract website articles, but YouTube subtitle extraction will not work.

Also, the Client Frontend and the Client Frontend Config both start a console window that does not close automatically when you close the browser window. You need to close those windows manually too. Sorry for the inconvenience there, but I have not found a reliable way to auto-close the server when the browser tab gets closed.

3. The Bundled Demo

The release contains a small default demo corpus:

```
SampleDB\Consciousness Travel - Default Demo
```

The demo DB is:

```
SampleDB\Consciousness Travel - Default Demo\Corpus\corpus.db
```

Fresh Frontend configs should point to that DB by default. If they do not, open the config page and set the SQL database path to the demo DB manually.

The demo is intentionally simple: one prepared corpus, already materialized, with evidence artifacts and a DB that can be queried immediately once the LLM provider is configured.

4. What To Configure

Open the config page and check two things first:

SQL database path
Ontology Machine root

For the demo, they should point to the bundled `corpus.db` and to the root folder of The Ontology Machine.

Then configure models.

LLM credentials unlock the agents:

- Query Agent answers
- Ontology Agent lens work
- Taxonomy Agent workflow guidance
- Kernel workflows that need model-assisted steps

Embedding credentials unlock vector retrieval:

- semantic search
- corpus embedding regeneration
- ontology embedding refresh

Missing embeddings are not corruption. The DB can still be valid. It just means semantic/vector search is weaker until embeddings are generated.

5. The Three Main Surfaces

The Machine has three normal working surfaces.

Surface	Start It	Use It For
Client Frontend Config	<code>Client Frontend\config.bat</code>	DB path, root path, credentials, models, prompts
Client Frontend Chat	<code>Client Frontend\start.bat</code>	Query Agent, Ontology Agent, Taxonomy Agent, sources, page viewer
Orchestrator	<code>00 - Orchestrator\run.bat</code>	Direct pipeline runs, ingestion debugging, stage output, artifact handling

There is also the Edit Suite, but it is not the first-stop UI. Use it later when you intentionally want owner-level editing and inspection.

6. Optional Extractor Tools

The release also contains a small helper folder:

```
Extractor_Tools\
```

These tools are not part of the core Machine. They are input-preparation helpers. Their job is simple: take material from the outside world and turn it into clean files you can drop into an Artifact Tree `Input` folder.

Use them when the source you want to analyze is not already a normal local document.

Tool	Start It	Use It For
Article Archive Extractor	<code>Extractor_Tools\Article Archive Extractor\Start Article Archive Extractor.bat</code>	Extract article URLs into Markdown files
YouTube Transcript Extractor	<code>Extractor_Tools\YouTube Transcript Extractor\Start YouTube Transcript Extractor.bat</code>	Extract available YouTube subtitles into Markdown files
Audio Transcription Extractor	<code>Extractor_Tools\Audio Transcription Extractor\Start Audio Transcription Extractor.bat</code>	Transcribe local audio/video files into Markdown files

The normal flow is:



Figure 2. Extractor flow: outside material becomes clean input for ingestion.

For example:

- use the Article Archive Extractor for news archives, web articles and online research material
- use the YouTube Transcript Extractor when a video already has subtitles
- use the Audio Transcription Extractor when you have a local media file and need a real transcript first

The extractor output is just input material. The Machine still does the actual rendering, extraction, validation, normalization, DB materialization and evidence handling during ingestion.

7. Which Agent Do I Use?

Use the Query Agent when you want to ask the existing DB questions.

Good first prompts:

Give me a compact coverage snapshot of this corpus.

What source documents are in this DB, and how complete is the page coverage?

Which ontology lenses exist, and what do they add?

Use the Ontology Agent when you want to build meaning on top of an existing DB.

Good first prompts:

Explain what ontology lenses would be useful for this corpus.

Create a review lens for weak or suspicious materialized facts.

Compare the active lens with the base corpus facts.

Behind the scenes, the Ontology Agent should inspect documents progressively: summary first, ontology evidence or rows next, full document only if the compact views are not enough. You do not need to ask for those tool names directly, but it explains why the Agent may do several background reads before it writes.

Use the Taxonomy Agent when you want Kernel-guided workflows.

Good first prompts:

What Kernel workflow state is active right now?

Create a new corpus DB with the default semantic release.

Run ingestion for the selected Artifact Tree.

Rebuild this DB from its Artifact Tree.

The Taxonomy Agent is not supposed to improvise a long chain of file edits in chat. It selects a Kernel workflow, the Kernel owns the state and dialogs, and the owner modules do the actual work.

8. When To Use The Orchestrator

Use the Orchestrator when you want the pipeline in your hands.

It is the direct control surface for the document mainline:

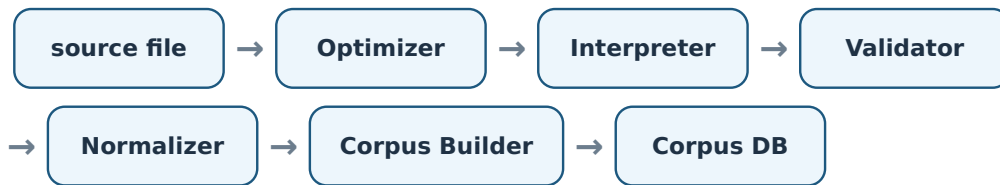


Figure 3. Direct document mainline through the Orchestrator.

The Orchestrator is the right choice when:

- you want to run ingestion directly
- you want to see pipeline stages
- you want to debug a bad input file
- you want to inspect stage artifacts
- you want to see why a document became an Error Case
- you want control without going through the Taxonomy Agent

Start it with:

```
00 - Orchestrator\run.bat
```

The Orchestrator and the Frontend use separate credential/config surfaces. If the Frontend says `LLM ready`, that does not automatically prove the pipeline owner credentials are configured for Orchestrator runs.

9. The Artifact Tree In One Minute

An Artifact Tree is the working folder around a corpus.

The normal shape is:

```
Artifact Tree\  
  Input\  
  Corpus\  
  Documents\  
  Error Cases\  
  Semantic Release\  
  
```

Input is where source files enter.

Corpus contains the SQLite DB.

Documents contains successful artifacts: originals, page images, raw extracts, model requests, structured outputs, validation reports, normalized outputs and logs.

Error Cases contains frozen diagnostic bundles. Do not treat them as trash. They are how the Machine makes failure visible enough to inspect.

Semantic Release contains the taxonomy/projection contract that shaped the normalized output and DB materialization.

If you remember only one thing: the Artifact Tree is the evidence surface. It lets you look behind an answer.

10. The Corpus DB In One Minute

The Corpus DB is a SQLite database. It is the thing the Query Agent and Ontology Agent read.

It contains more than text chunks:

- document and page records
- source-document grouping
- page images for evidence back-linking
- extracted fields and rows
- promoted semantic values
- evidence atoms
- embeddings when available
- Base Graph relations
- ontology lenses

The DB is not supposed to replace the Artifact Tree. The DB is the queryable materialization. The Artifact Tree remains the rebuild and audit surface.

11. What Is A Semantic Release?

A Semantic Release is the active meaning contract of a corpus.

It defines which taxonomy and projection the Normalizer uses. Without an active Semantic Release, ingestion has no stable target language and the Corpus Builder should refuse to materialize new documents.

In normal use, you do not hand-build this by copying files around. Either the Orchestrator route or the Taxonomy Agent/Kernel route creates and binds the needed pieces.

12. What Is An Ontology Lens?

The corpus DB contains the materialized facts.

An ontology lens is a named perspective over those facts.

It can be:

- a reading lens
- a review lens
- a correction lens
- a domain model
- a relationship graph
- a working theory
- a peer-review layer

The key idea is safety: the Ontology Agent does not overwrite the base corpus truth. It adds evidence-bound lenses that can extend, question or reinterpret the base layer while keeping the difference visible.

That is why the same corpus can be read in different ways without rebuilding the whole DB.

13. Create Your Own Corpus

There are two sane routes.

Route A: Direct Orchestrator Run

Use this when you want visible pipeline control.

1. Start `00 - Orchestrator\run.bat`.
2. Create or select the Artifact Tree target.
3. Make sure the workflow has a Corpus DB and active Semantic Release.
4. Put source files into `Input` or select them through the UI.
5. Run the pipeline.
6. Inspect `Documents` and `Error Cases`.
7. Open the resulting DB in the Frontend.

This route is best for pipeline debugging and document-stage inspection.

Route B: Taxonomy Agent / Kernel Workflow

Use this when you want the guided workflow.

The Kernel can create the Artifact Tree, create the Corpus DB, attach a default or custom Semantic Release, ask for needed paths through UI dialogs, and then run ingestion through the owner modules.

1. Open the Client Frontend.
2. Select the Taxonomy Agent.
3. Pick a workflow from the workflow dropdown or ask for one in chat.
4. Answer Kernel dialogs in the UI.
5. Let the workflow finish with a final notice.
6. Query the resulting DB.

This route is best for normal guided creation, rebuild, merge and recovery.

14. First Useful Demo Session

After the demo DB and LLM credentials are configured, try this flow:

1. Ask the Query Agent for a coverage snapshot.
2. Ask which source document is present and how many pages were materialized.
3. Ask whether a Base Graph exists.
4. Ask which ontology lenses exist.
5. Ask one conceptual question about the corpus.
6. Open a cited source in the source viewer.
7. Ask the Ontology Agent what additional lens would make sense.
8. Ask the Taxonomy Agent for current Kernel state.

That gives you the whole product shape without building a new corpus first.

15. Common First Problems

Failed to fetch

Usually the chat server is not running. Start:

```
Client Frontend\start.bat
```

If only the config page is open, the agents are not running.

Browser Does Not Open

Read the console output and open the printed URL manually.

Normal URLs:

```
http://127.0.0.1:3000  
http://127.0.0.1:3001/config
```

Query Agent Says No DB Is Configured

Open config and set the SQL database path to an existing corpus DB.

For the bundled demo:

```
SampleDB\Consciousness Travel - Default Demo\Corpus\corpus.db
```

Pipeline Says No Active Semantic Release Exists

The selected target is not ready for ingestion. Create or apply a Semantic Release through the Orchestrator or Taxonomy Agent workflow. Do not fix this by manually editing random DB fields.

Embeddings Are Missing

Configure an embedding provider and regenerate embeddings if you need vector search. Until then, SQL and lexical retrieval can still work.

Error Cases Appear

Open the `Error Cases` folder and inspect the bundle. The point of Error Cases is not to pretend the Machine never fails. The point is to make failure visible and useful to inspect.

A Kernel Workflow Looks Stuck

Ask the Taxonomy Agent for Kernel state or use the visible workflow controls. Do not start the same workflow again over the same target until the old state is completed, cancelled or clearly blocked.

16. What To Read Next

This Quickstart is only the front door.

Read next:

- `01_System_Overview.md` for the complete system shape
- `06_Artifact_Tree_Guide.md` for the evidence folder structure
- `07_Database_Documentation.md` for the DB schema and data layers
- `08_Agent_Documentation.md` for Query, Ontology and Taxonomy Agent behavior
- `09_Configuration_Credentials.md` for providers, credentials and config
- `10_Operator_Guides.md` for real operating procedures

Start small. Open the demo. Ask questions. Follow the sources. Then build your own corpus once the basic shape makes sense.

Final rule

The quickstart in one line

Start small. Open the demo, ask questions, follow the sources, and build your own corpus only after the basic evidence model is clear.